Week 4 · Lesson 3
NER with Transformers

Practical exercise | Approximately 60 minutes

You will configure a pretrained named-entity recognition (NER) pipeline, run batched inference on 20 sentences, inspect subtoken aggregation, and export character-aligned predictions as JSONL for later evaluation. You will use an existing fine-tuned model; you will not train it in this exercise.

Create your notebook. Open Google Colab, select **File → New notebook**, and name it **week4_lesson3_ner.ipynb**. Add one code cell for each numbered code step (1-7) below and run the cells in order. Add a final markdown cell for your reflection. No starter notebook is supplied.

Runtime: Internet access is required for installation and the first model download (about 1.3 GB). A GPU is optional and helpful; CPU inference is slower. Start with a fresh runtime before Step 1.

1. Set up the environment

Copy this into your first code cell. **json**, **time**, and **pathlib** are Python standard-library modules; do not install them with pip.

```
%pip install -q "transformers==4.57.6" "torch>=2.6,<3" pandas

import os
os.environ["USE_TF"] = "0"
os.environ["USE_FLAX"] = "0"
import json
import time
import torch
import pandas as pd
from pathlib import Path
from IPython.display import display
from transformers import pipeline

DEVICE = 0 if torch.cuda.is_available() else -1
BATCH_SIZE, THRESHOLD = 8, 0.5
torch.set_num_threads(2)
print("Device:", "GPU" if DEVICE == 0 else "CPU")
```

2. Load the NER pipeline

```
MODEL_ID = (
    "dbmdz/bert-large-cased-finetuned-conll03-english"
)
ner = pipeline(
    "token-classification", model=MODEL_ID,
    aggregation_strategy="simple", use_fast=True,
    framework="pt", device=DEVICE, dtype=torch.float32
)
assert ner.tokenizer.is_fast
print("Checkpoint:", MODEL_ID)
print("Token labels:", ner.model.config.id2label)
test_text = "Barack Obama was born in Honolulu on August 4, 1961."
print(ner(test_text))
```

Read the labels: PER = person, ORG = organization, LOC = location, and MISC = miscellaneous. B-/I- mark entity beginnings/continuations; O means outside an entity. This checkpoint has no DATE or MONEY labels. Do not expect it to extract the date in the test sentence. [1]

Prepare inputs and measure inference

3. Define and validate your sample texts

Use these 20 sentences, or replace them with 20-30 short English sentences. Preserve their case, punctuation, and accents: offsets must refer to the unchanged input. The length check includes special tokens and prevents unnoticed truncation of long inputs.

```
# These illustrative sentences are inputs, not gold annotations.
samples = [
    "Google opened an office in New York City in October 2021.",
    "The FDA reviewed a report from Moderna in December 2020.",
    "Amazon's team met in Virginia before visiting New York.",
    "Barack Obama spoke at an event in Chicago.",
    "Microsoft hired engineers in London and Dublin.",
    "Apple's supplier sent a shipment to Berlin.",
    "Marie Curie studied in Paris.",
    "The United Nations held a meeting in Geneva.",
    "Toyota announced a partnership with a company in Tokyo.",
    "Liverpool played against Manchester United on Saturday.",
    "A French journalist interviewed Angela Merkel in Brussels.",
    "The World Health Organization published a report.",
    "Jean-Pierre works for a start-up in Lyon.",
    "O'Connor visited St. Mary's Hospital in London.",
    "José García attended a conference in Madrid.",
    "Beyoncé performed in Los Angeles.",
    "The European Commission met representatives from Germany.",
    "IBM and Samsung discussed a project in Seoul.",
    "The shipment cost $500 and arrived on 12 June 2022.",
    "The team discussed the report after lunch.",
]
assert 20 <= len(samples) <= 30
token_ids = ner.tokenizer(
    samples, truncation=False, verbose=False
)["input_ids"]
limit = ner.model.config.max_position_embeddings
assert max(map(len, token_ids)) <= limit, "Shorten long inputs."
print("Sentences:", len(samples), "Token limit:", limit)
```

4. Run batched inference and report timing

```
def synchronize():
    if DEVICE == 0:
        torch.cuda.synchronize()

# Warm up on one batch; exclude model loading and warm-up.
ner(samples[:BATCH_SIZE], batch_size=BATCH_SIZE, num_workers=0)
synchronize()
start = time.perf_counter()
all_preds = ner(samples, batch_size=BATCH_SIZE, num_workers=0)
synchronize()
elapsed = time.perf_counter() - start
assert len(all_preds) == len(samples)
print(f"Batch size: {BATCH_SIZE}; total time: {elapsed:.2f} s")
print(f"Throughput: {len(samples) / elapsed:.2f} sentences/s")
print(f"Amortized time: {1000 * elapsed / len(samples):.2f} ms/sentence")
```

Interpret timing correctly. This is one warmed-up batch-processing run. Time divided by sentence count is amortized time, not the response latency of one isolated request. It includes tokenization and pipeline processing, but excludes loading and warm-up. Record the device and batch size. Results will vary across hardware and runs; this small sample is not a production benchmark.

Create and verify structured output

5. Filter scores and preserve exact spans

The pipeline groups compatible B-/I- predictions with **aggregation_strategy="simple"**. Grouping can still produce partial or incorrect spans. Use **text[start:end]** rather than the reconstructed **word** field to store the exact source text. [2]

```
records = []
for i, (text, preds) in enumerate(zip(samples, all_preds), start=1):
    rec = {
        "doc_id": f"doc_{i:03d}", "text": text,
        "model": MODEL_ID, "threshold": THRESHOLD,
        "predictions": []
    }
    for e in preds:
        score = float(e["score"])
        if score < THRESHOLD:
            continue
        start, end = int(e["start"]), int(e["end"])
        assert 0 <= start < end <= len(text)
        rec["predictions"].append({
            "entity": e["entity_group"],
            "start": start, "end": end,
            "text": text[start:end], "score": score
        })
    records.append(rec)

before = sum(len(preds) for preds in all_preds)
after = sum(len(rec["predictions"]) for rec in records)
print(f"Spans before filtering: {before}; retained: {after}")
print(json.dumps(records[0], ensure_ascii=False, indent=2))
```

Offset convention: start is inclusive and end is exclusive, using zero-based Python string character indices, not byte offsets. The 0.5 threshold is illustrative; a score is not a calibrated guarantee of correctness. Keep full-precision scores for later analysis.

6. Save UTF-8 JSONL and verify a round trip

Write one JSON object per line, including sentences with no retained entities. Stable document IDs and original text will help you match predictions to future gold annotations.

```
output_path = Path("ner_output.jsonl")
with output_path.open("w", encoding="utf-8") as f:
    for rec in records:
        f.write(json.dumps(rec, ensure_ascii=False) + "\n")

with output_path.open(encoding="utf-8") as f:
    loaded = [json.loads(line) for line in f]
assert loaded == records
assert len(loaded) == len(samples)
assert len({rec["doc_id"] for rec in loaded}) == len(samples)
for rec in loaded:
    for e in rec["predictions"]:
        assert e["text"] == rec["text"][e["start"]:e["end"]]
        assert THRESHOLD <= e["score"] <= 1.0
df = pd.read_json(output_path, lines=True, encoding="utf-8")
display(df[["doc_id", "text", "predictions"]].head())
print("Verified JSONL:", output_path.resolve())
```

Expected result: one record for each sentence, a five-row DataFrame preview, and a printed output path. Every saved span must equal the corresponding source-text slice. The assertions check data integrity; they do not establish that the entity labels are correct.

Inspect predictions and reflect

7. Inspect subtokens and challenging spans

Compare the raw token predictions with the grouped spans. Look for split names, incomplete boundaries, missed entities, and incorrect labels. If you changed the sample list, update the indices to point to suitable examples.

```
# Compare token-level predictions with grouped spans.
idx = 12  # Jean-Pierre: inspect a hyphenated name.
print("Input:", samples[idx])
raw = ner(samples[idx], aggregation_strategy="none")
display(pd.DataFrame(raw)[
    ["word", "entity", "start", "end", "score"]
])
print("Retained grouped spans:")
print(json.dumps(records[idx]["predictions"],
                 ensure_ascii=False, indent=2))

# Inspect an apostrophe and an accented name as well.
for idx in (13, 14):
    print("\nInput:", samples[idx])
    print(json.dumps(records[idx]["predictions"],
                     ensure_ascii=False, indent=2))
```

Do not delete all but the first subtoken or force boundaries to full words: that would change the model output and could hide errors. The first-subtoken labeling convention sometimes used during training is a separate issue. Inspect at least one hyphenated or apostrophe-containing example and one sentence without predicted entities.

8. Write your reflection

In your final markdown cell, write **3-5 sentences** that address all of the following: (1) quote one convincing prediction and its entity type; (2) identify one concrete miss, false positive, or boundary error; (3) explain a limitation of the label set or the filtering threshold; (4) propose one adjustment and explain how you would validate it. Use evidence from your own outputs. If your selected examples show no clear error, add a challenging sentence and rerun Steps 3-7.

Before formal evaluation: You need manually verified gold entities for these same texts, with the same label set and offset convention. Span-level precision, recall, and F1 require comparing predicted and gold entity types and boundaries. JSONL predictions alone cannot provide these metrics. If you tune the threshold, use separate validation data and reserve the test data for final evaluation.

What you will submit

Submit your **executed .ipynb notebook** with the checkpoint labels, timing results, JSONL preview and verification, inspected predictions, and final reflection. Keep **ner_output.jsonl** for the later evaluation activity. In Colab, download the notebook through **File → Download → Download .ipynb**; download the JSONL from the Files panel before the runtime is reset.

Completion check

Restart the runtime and run all cells from the beginning. Confirm that there are no errors, the number of JSONL records equals the number of sentences, and all span checks pass. Expected entity predictions may vary; an empty predictions list is valid.

Technical references

[1] dbmdz. BERT-large cased, fine-tuned on CoNLL-03 English, model configuration (id2label). Checkpoint and configuration.
[2] Hugging Face. Transformers v4.57.1: TokenClassificationPipeline (aggregation strategies and character offsets). Pipeline documentation.